MOBILE APPLICATION DEVELOPMENT

Bachelor's Degree Group Project Assignment

Java-based Android Applications with REST API, Remote Database, Authentication & Software Architecture

1. Project Overview

This group project requires students to design, implement and document a complete mobile information system. Each group builds a distinct Android application in Java that communicates with a backend REST API and a remote database. The assignment emphasises software engineering practices appropriate for Bachelor's-level study: requirements engineering, system architecture, data modelling, secure authentication, API design, testing, version control and professional documentation.

All groups must deliver a working signed APK, source code hosted on GitHub with individual commit history, a REST API backend, remote database, and comprehensive project documentation.

2. Group Allocations & Assigned Applications

Group	Students	Application	Scope
1	Athens Gandize Atupele Beza Benjamin Lungu	**Student Attendance & Academic Tracker** QR attendance, role-based access, analytics, offline sync	Full (3)
2	Cassius Mapalo Glory Mwiti Harvey Lusambo	**Small Business Sales & Inventory** Roles, sales, stock, reports, PDF receipts, offline mode	Full (3)
3	Mainza Linyando Michelle Bupe Mwala Mutale Chewe	**Campus Event & Notice Board** Admin, registration, QR check-in, maps, notifications	Full (3)
4	Naomi Akilimali Rabecca Chola	**Personal Expense & Budget Manager** Auth, budgets, analytics, charts, export, offline sync	Focused (2)

Note: Group 4 has a slightly reduced feature set to match the smaller team size while still meeting the same technical depth requirements.

3. Mandatory Technical Requirements (All Groups)

Every project must demonstrate the following capabilities. These raise the assignment to Bachelor's standard:

- **Android Client (Java):** Activities, Fragments, RecyclerView, Material Design, navigation component or equivalent, proper lifecycle handling.
- **Architecture:** MVC, MVP or MVVM (or clean architecture). Separation of UI, business logic and data layers.
- **Local Storage:** SQLite or Room for offline capability and caching.
- **Remote Backend:** REST API (any reasonable stack: Spring Boot, Node/Express, Flask, etc.) with proper endpoints.
- **Remote Database:** MySQL, PostgreSQL or equivalent. Normalised schema with relationships.
- **Authentication & Authorization:** Secure login (JWT or session-based). Role-based access control where applicable.
- **Networking:** HTTP client (Retrofit / Volley / OkHttp), JSON parsing, proper error handling and connectivity checks.
- **Offline & Sync:** Core operations must work offline; changes synchronise when connectivity returns.
- **Security:** Input validation, secure password handling, HTTPS where possible, protection against common mobile threats.

- **Version Control:** Git/GitHub repository. Every student must have identifiable commits throughout the project.
- **Testing:** Unit tests for business logic, basic instrumentation/UI tests, and a written test report.
- **Documentation:** Requirements specification, architecture diagram, ER diagram, API documentation, user manual, and final report.

4. Detailed Project Specifications

Group 1 — Student Attendance & Academic Tracker

Students: Athens Gandize, Atupele Beza, Benjamin Lungu

Build a complete attendance and academic tracking system for a university setting with lecturer and student roles.

Core Features (Mandatory)

- Student and lecturer registration / authentication with role-based access
- Course registration and assignment of students to courses
- Attendance recording by lecturers (manual + QR-code scanning)
- Student view of personal attendance history and percentage
- Lecturer dashboard with class statistics and absence alerts
- Search and filter students / courses
- Attendance percentage calculation and reporting
- Local SQLite/Room cache + REST API + remote database
- Offline attendance recording with later synchronisation
- Notifications / reminders for low attendance
- Input validation and exception handling throughout

Expected Advanced Elements

- QR code generation and scanning for attendance
- Meaningful analytics (attendance trends, class averages)
- Role-based dashboards (different screens for lecturer vs student)

Group 2 — Small Business Sales & Inventory App

Students: Cassius Mapalo, Glory Mwiti, Harvey Lusambo

Design a realistic point-of-sale and inventory system for a small retail business with administrator and cashier roles.

Core Features (Mandatory)

- User authentication with Administrator and Cashier roles
- Product management (CRUD, categories, prices, stock quantities)
- Sales processing: select products, calculate total, reduce stock automatically
- Generate and display/share simple PDF or text receipts
- Inventory views, low-stock alerts and stock-in / stock-out operations
- Sales reports: daily, weekly, monthly; best-selling products; total revenue
- Search and filtering of products and sales
- REST API + remote database + local Room/SQLite cache
- Offline sales capability with synchronisation when online
- Notifications for low stock
- Proper data validation and error handling

Expected Advanced Elements

- PDF receipt generation
- Revenue and inventory analytics dashboards
- Role-restricted functionality (cashiers cannot delete products, etc.)

Group 3 — Campus Event & Notice Board App

Students: Mainza Linyando, Michelle Bupe Mwala, Mutale Chewe

Create a campus-wide event and announcement platform supporting student and administrator roles.

Core Features (Mandatory)

- Student and administrator authentication with role-based access
- View, create, edit and delete announcements and events (admin)
- Event details, categories, search and filtering
- Student event registration / RSVP
- QR-code based event check-in
- Push or local notifications and event reminders
- Image upload for event posters (or links to images)
- Basic location / map integration for event venues (optional but recommended)
- REST API + remote database + local caching
- Offline viewing of previously loaded events with later sync
- User profiles and attendance / registration statistics

Expected Advanced Elements

- QR check-in system
- Image handling and notifications
- Map / location awareness for events

Group 4 — Personal Expense & Budget Manager

Students: Naomi Akilimali, Rabecca Chola

Develop a personal finance application focused on depth of analytics and clean architecture rather than breadth of features.

Core Features (Mandatory)

- User authentication and profile management
- Add, edit, delete income and expense transactions
- Expense categories and monthly budget setting
- Real-time remaining budget calculation and spending alerts
- Transaction history with search and filtering
- Monthly and category-based spending summaries
- Graphical reports / charts (bar, pie or line)
- REST API + remote database + local Room/SQLite storage
- Offline transaction entry with synchronisation
- Data export (CSV or PDF summary)
- Input validation, calculations and notifications

Expected Advanced Elements

- Meaningful data visualisation (charts)
- Budget alerts and analytics
- Export functionality

5. Seven-Week Incremental Schedule

The project is deliberately incremental. Each week builds on the previous one and produces a graded deliverable. This structure ensures continuous participation and prevents last-minute work by a single student.

Week 1 — Requirements Engineering

- Problem statement and stakeholder analysis
- Functional and non-functional requirements
- Use-case diagrams and prioritised feature list
- High-level system context diagram
- Team roles and individual responsibility matrix
- GitHub repository created; all members added

Deliverable: Project Proposal Document + GitHub link

Week 2 — System & Database Design

- Application architecture (MVC/MVP/MVVM or clean)
- Entity-Relationship diagram and normalised database schema
- REST API endpoint design (resources, methods, sample payloads)
- Authentication and role model design
- Sequence diagrams for key flows (login, main transaction)

Deliverable: System Design Document (architecture + ER + API sketch)

Week 3 — UI/UX & Android Foundation

- Wireframes / high-fidelity mockups for all major screens
- Navigation flow diagram
- XML layouts implemented (Material Design preferred)
- Activities, Fragments, basic navigation working
- Splash, login/register, home/dashboard screens functional (mocked data)

Deliverable: UI/UX package + working Android shell (no backend yet)

Week 4 — Core Application Logic & Local Storage

- Java business logic, validation, calculations
- Room or SQLite entities, DAOs and repositories
- CRUD operations working offline
- RecyclerViews, adapters, forms and dialogs complete
- Basic authentication UI fully wired to local data

Deliverable: Functional offline prototype with local database

Week 5 — Backend, API Integration & Sync

- REST API implemented and deployed (or runnable locally)
- Remote database populated and accessible
- Android client consumes API (Retrofit/Volley/etc.)
- Authentication tokens / sessions working end-to-end
- Offline-to-online synchronisation implemented
- Error handling for network failures

Deliverable: End-to-end working system (Android ↔ API ↔ DB)

Week 6 — Advanced Features, Testing & Security

- Group-specific advanced features (QR, charts, PDF, maps, etc.)
- Unit tests for critical business logic
- UI / instrumentation tests for main flows
- Security review: validation, auth, data protection

- Performance and usability improvements
- Test report written

Deliverable: Test Report + feature-complete application

Week 7 — Deployment, Documentation & Presentation

- Signed release APK generated
- Final documentation (requirements, architecture, ER, API docs, user manual)
- GitHub repository cleaned and README complete
- Individual contribution statements prepared
- Live demonstration and technical defence

Deliverable: Final package: APK + source + docs + presentation

6. Assessment Breakdown (100 Marks)

Component	Marks	Notes
Project Proposal (Week 1)	8	Requirements quality, clarity, feasibility
System & Database Design (Week 2)	10	Architecture, ER model, API design
UI/UX & Android Foundation (Week 3)	10	Design quality, navigation, Material Design
Core Logic + Local DB (Week 4)	12	Java quality, Room/SQLite, offline CRUD
Backend + API Integration (Week 5)	15	Working REST API, auth, sync, error handling
Advanced Features + Testing (Week 6)	12	Group-specific features, test coverage & report
Documentation & Final Package	10	Completeness, professionalism, user manual
Final Presentation & Demo	10	Live demo, technical questions, defence
Individual Contribution & Git	13	Personal code ownership, commit history, explanation

Individual contribution is assessed by examining each student's Git commits and by asking them to explain or modify the code they claim ownership of during the final defence.

7. Suggested Team Roles (Rotate)

Roles should rotate across the seven weeks so that every student gains experience in UI, business logic, data layer and testing.

Role	Primary Responsibilities
UI / Frontend	XML layouts, Activities, Fragments, Navigation, Material Design, RecyclerViews, user experience
Application Logic	Java classes, business rules, validation, calculations, ViewModels / presenters, architecture
Data & Backend	Room/SQLite, REST API design & implementation, remote database, authentication, synchronisation
Quality & Integration	Testing, debugging, Git hygiene, documentation, integration of components, final packaging

For the 2-person group, each student covers two role areas and both must contribute to backend and testing.

8. Individual Participation Rules

To prevent free-riding, the following rules apply:

- Every student must make regular, identifiable commits to the shared GitHub repository.
- Each student must be able to explain and modify the modules they developed.
- During the final presentation the examiner may ask any student to demonstrate or alter a specific part of the code.
- A student who cannot demonstrate meaningful technical contribution may receive a significantly lower individual mark even if the group product is strong.

• Weekly deliverables are graded; missing a weekly submission affects the whole group but individual effort is still tracked via Git.

9. Final Submission Checklist

• Signed release APK (installable on a physical device or emulator)
• Complete source code (Android + backend) on GitHub with clear README
• Database schema and sample data scripts
• API documentation (endpoints, methods, request/response examples)
• Project documentation: requirements, architecture diagrams, ER diagram, design decisions
• User manual (screenshots + step-by-step usage)
• Test report (unit + UI tests + results)
• Individual contribution statements (what each member developed)
• Presentation slides
• Live demonstration on the day of assessment

This assignment is designed to give Bachelor's students practical experience of the full mobile application lifecycle: from requirements and architecture through to a deployed, tested, documented system that uses modern Android practices, a real backend, and professional collaboration tools.

Questions about scope or technical choices should be raised with the lecturer early. Good luck.